

Програмування мовою C++. Структури та об'єднання. Перерахування.

1. Структури в C++

1.1. Поняття структури

Структура — це агрегатний тип даних, що об'єднує кілька змінних різних типів під одним ім'ям.

Поле структури — окрема змінна всередині структури.

2. Типи структур у середовищі CLR (C++/CLI)

Існують два види:

2.1. Native-структури (C/C++)

- Класичні структури.
- Розміщуються в стеку або статичній пам'яті.
- Передаються у функції:
 - ✓ за значенням
 - ✓ за адресою (через *)

2.2. Managed-структури

Оголошуються з кваліфікаторами:

Кваліфікатор	Тип	Де розміщується	Як передається
ref struct	референсний тип	керована heap	передається як посилання ^
value struct	тип-значення	стек або heap в коробці	як значення або через *

3. Оголошення структур

3.1. Native-структура

```
struct Worker {  
    int code;  
    char name[50];  
    int hours;  
    float cost;  
};
```

3.2. Managed ref-структура

```
ref struct MyPointRef {  
    int x;  
    int y;  
};
```

3.3. Managed value-структура

```
value struct MyPointValue {  
    int x;  
    int y;  
};
```

4. Створення структурних змінних

Спосіб 1 — після шаблону

```
Worker w;
```

Спосіб 2 — під час оголошення

```
struct Worker { ... } w1;
```

Спосіб 3 — через typedef

```
typedef struct { int code; ... } WORKER;  
WORKER w2;
```

5. Доступ до полів

Якщо структура — змінна:

w.code
w.hours

Якщо структура — вказівник:

p->code
(*p).hours

6. Ініціалізація структур

Worker w = {150, "Smith", 40, 20.5};

7. Передача структур у функції

7.1. Native-структура

Передача за значенням

Копія → зміни не впливають на оригінал.

bool Equal(MyPoint p1, MyPoint p2);

Передача за адресою

Швидко, але можна змінити оригінал.

bool Equal(MyPoint *p1, MyPoint *p2);

7.2. Managed ref-структури (^)

Передаються тільки як посилання:

float LengthRef(MyPointRef ^p1, MyPointRef ^p2);

7.3. Managed value-структури

Два способи:

1) За значенням

```
float LengthValue(MyPointValue p1, MyPointValue p2);
```

2) Через покажчик

```
float LengthValueP(MyPointValue *p1, MyPointValue *p2);
```

8. Повернення структур з функцій

Варіанти:

1. За значенням

2. Покажчиком

3. Для ref/value структур

залежить від їхнього типу (^, * або значення).

9. Структури і функції (приклад)

У файлі був приклад структури телефонної книги:

```
struct Phone_directory {  
    string name;  
    string phone;  
};
```

Функції:

- запис у файл
- читання з файлу
- передача структури у функцію **за значенням**

10. Об'єднання (union)

10.1. Поняття

Об'єднання — це тип даних, в якому **всі поля займають одну й ту ж пам'ять**, тобто можуть існувати тільки по одному.

Оголошення:

```
union Sample {  
    int x;  
    float y;  
    char z;  
};
```

Займає стільки байтів, скільки найбільше поле.

11. Перерахування (enum)

11.1. Поняття

Перелічуваний тип — це набір символічних констант.

```
enum Color { Red, Green, Blue };
```

11.2. Значення

За замовчуванням:

- Red = 0
- Green = 1
- Blue = 2

Можна задавати вручну:

```
enum Day { Mon=1, Tue=2, Wed=3 };
```

12. Обробка перерахувань

```
Color c = Blue;  
if (c == Blue) ...
```

13. Вивід enum у консоль

Зазвичай через switch або власний масив назв.

14. Бітові поля в структурах

Дозволяють економити пам'ять.

```
struct Flags {  
    unsigned int a : 1;  
    unsigned int b : 3;  
    unsigned int c : 4;  
};
```

- a займає 1 біт
- b — 3 біти
- c — 4 біти

Використовуються для:

- прапорців
- компактного зберігання статусів
- роботи з портами вводу/виводу

15. Структури, спадковість, поліморфізм

У файлах пояснено, що:

- У C++ структури можуть мати **методи**, конструктори, функції-члени.
- За замовчуванням доступ публічний.
- Можуть **успадковуватися** одна від одної (як класи).
- Можуть мати **віртуальні функції** → поліморфізм.

Приклад:

```
struct Base { virtual void show(); };  
struct Der : Base { void show() override; };
```